

Report HW 5 ECE-111

Name: Jasper Huang

PID: A17796149

Homework 5a (Gray to Binary Code Converter)

1. SystemVerilog design code

a. gray_code_to_binary_converter

```
module gray_code_to_binary_converter #(parameter N = 4)(
    input logic clk, rstn,
    input logic[N-1:0] gray_value,
    output logic[N-1:0] binary_value);

    // Add code for gray code to binary conversion
    function automatic logic [N-1:0] gray_to_binary(input logic [N-1:0] value);
        gray_to_binary[N-1] = value[N-1];
        for (int i = N-2; i >= 0; i = i - 1)
            gray_to_binary[i] = gray_to_binary[i+1] ^ value[i];
    endfunction

    always_ff @(posedge clk or negedge rstn) begin
        if (!rstn) binary_value <= '0;
        else      binary_value <= gray_to_binary(gray_value);
    end

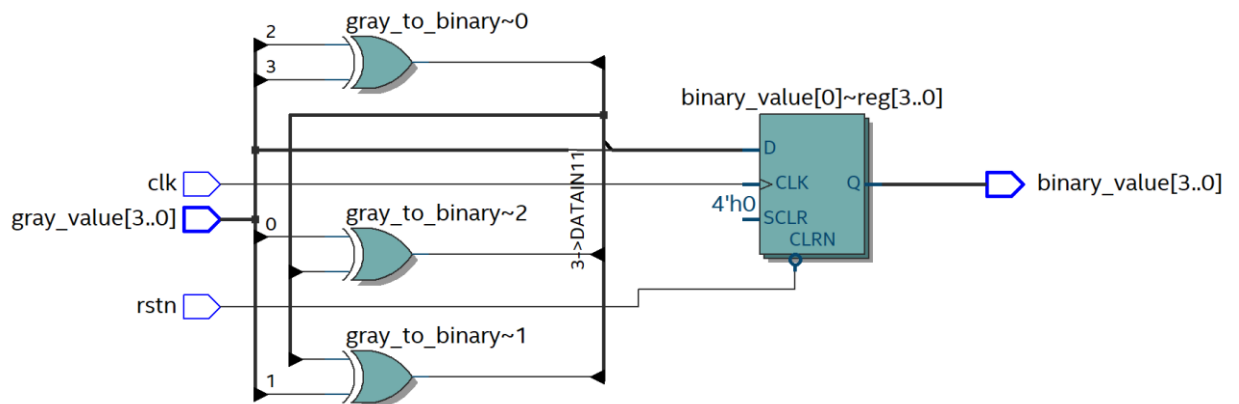
endmodule: gray_code_to_binary_converter
```

2. Synthesis Resource usage and schematic generated from RTL netlist viewer

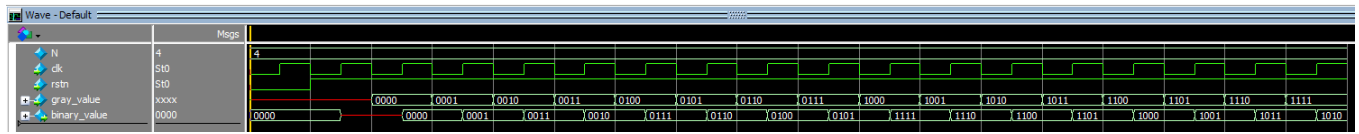
a. Resource Usage

Analysis & Synthesis Resource Usage Summary		
<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	2
2		
3	▼ Combinational ALUT usage for logic	3
1	-- 7 input functions	0
2	-- 6 input functions	0
3	-- 5 input functions	0
4	-- 4 input functions	1
5	-- <=3 input functions	2
4		
5	Dedicated logic registers	4
6		
7	I/O pins	10
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	gray_value[3]~input
12	Maximum fan-out	4
13	Total fan-out	35
14	Average fan-out	1.30

b. RTL netlist viewer

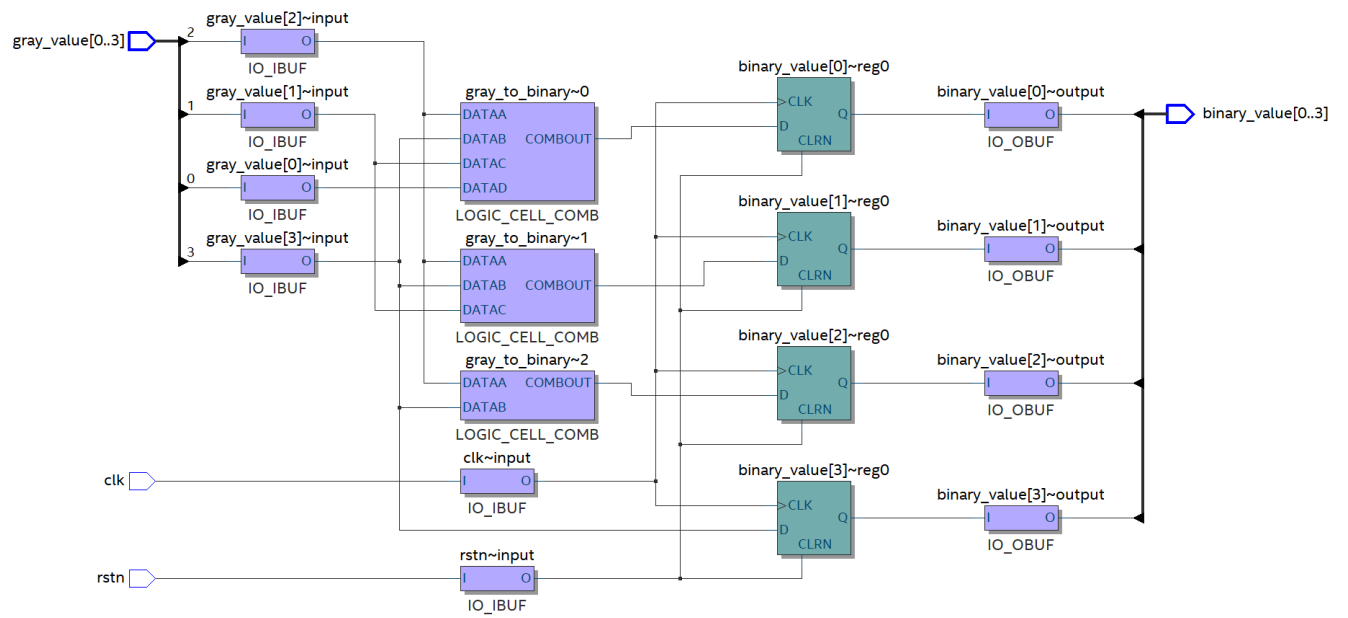


3. Simulation snapshot and explain simulation result



The simulation wave shows the conversion of gray value to binary value for all cases from 0000 to 1111 which output a correct binary value.

4. Post-Mapping schematic



Homework 5b (Carry Lookahead Adder)

1. SystemVerilog code snapshot

a. carry_lookahead_adder

```
module carry_lookahead_adder#(parameter N=4)(
    input logic[N-1:0] A, B,
    input logic CIN,
    output logic[N:0] result
);

    // Add code for carry lookahead adder
    wire [N-1:0] p = A ^ B;
    wire [N-1:0] g = A & B;

    wire [N:0] c;
    assign c[0] = CIN;
    genvar i;
    generate
        for (i = 0; i < N; i = i + 1) begin : carry_gen
            assign c[i+1] = g[i] | (p[i] & c[i]);
        end
    endgenerate

    wire [N-1:0] fa_cout;
    generate
        for (i = 0; i < N; i = i + 1) begin : fa_loop
            fulladder fa_inst (
                .a (A[i]),
                .b (B[i]),
                .cin (c[i]),
                .sum (result[i]),
                .cout(fa_cout[i])
            );
        end
    endgenerate

    assign result[N] = c[N];

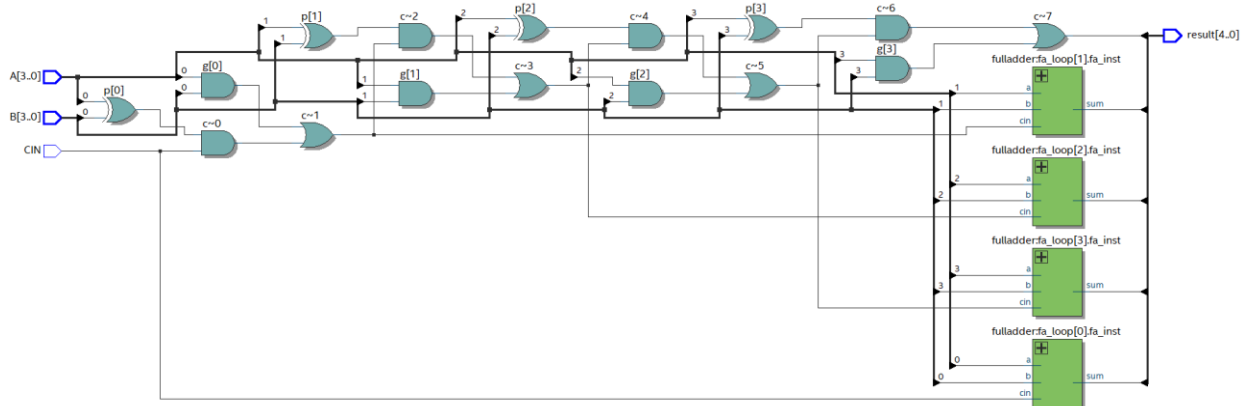
endmodule: carry_lookahead_adder
```

2. Synthesis Resource usage and schematic generated from RTL netlist viewer

a. Synthesis Resource usage

Analysis & Synthesis Resource Usage Summary		
<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	3
2		
3	▼ Combinational ALUT usage for logic	6
1	-- 7 input functions	0
2	-- 6 input functions	0
3	-- 5 input functions	4
4	-- 4 input functions	0
5	-- <=3 input functions	2
4		
5	Dedicated logic registers	0
6		
7	I/O pins	14
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	c[2]
12	Maximum fan-out	3
13	Total fan-out	45
14	Average fan-out	1.32

b. RTL netlist viewer

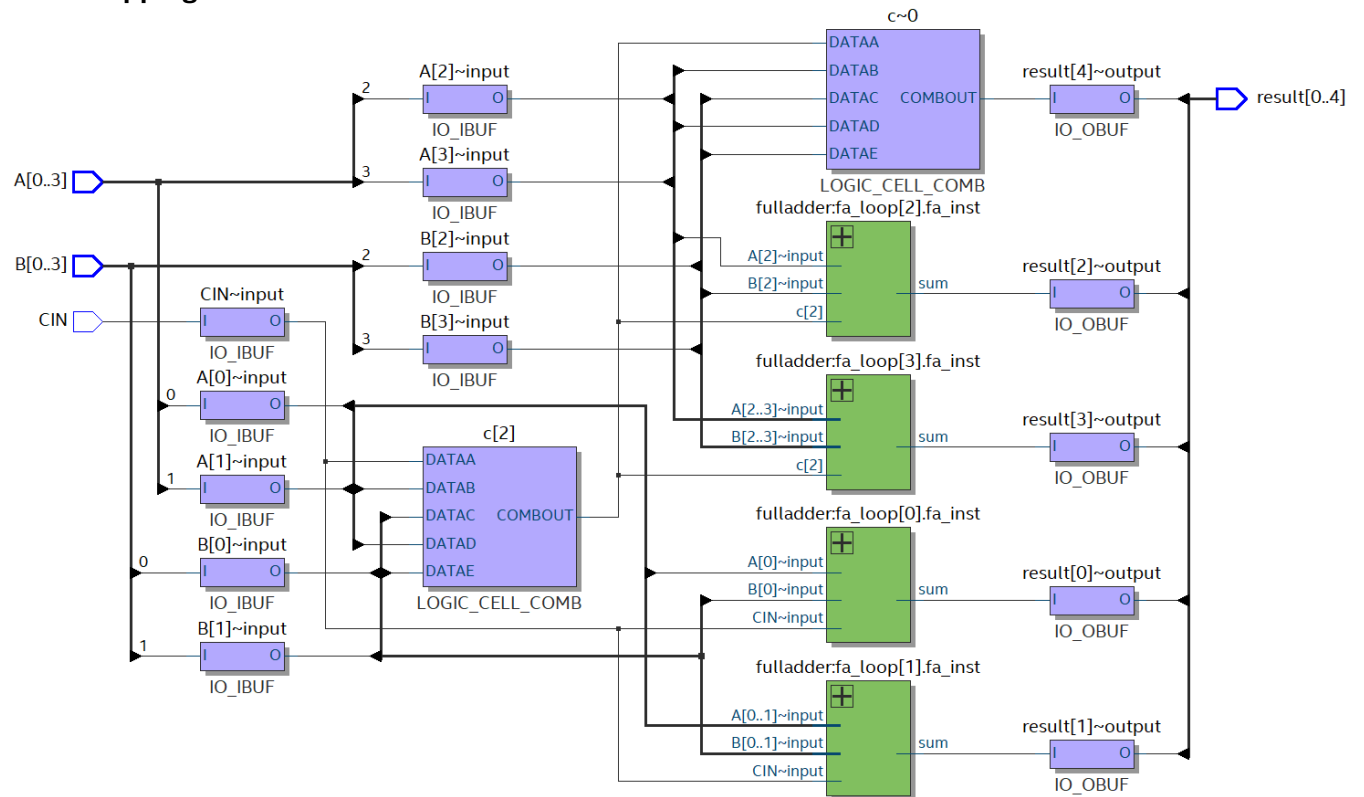


3. Simulation snapshot and explain simulation result

Wave - Default									
		Msgs							
N	4	4							
A	0	0	2	1	2	3	4	15	10
B	0	0	1		2	1	7	2	5
CIN	0								
result	0	0	4	2	4		12	17	15

The simulation wave shows a result of $A+B+CIN$ for each of the A and B and CIN value which output a correct result for each case.

4. Post-Mapping schematic

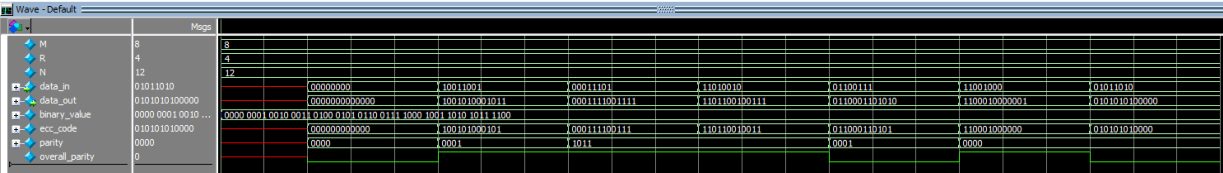


Homework 5c (ECC Encoder and ECC Decoder)

ECC Encoder

1. Simulation waveform

a.



2. Modelsim transcript snapshot

```
VSIM 11> run -all
# *Test Passed* : data_out generated from design matches data_out generated from tesbench!
# - At time=15 For Input data_in : 00000000, Expected data_out from testbench = 00000000000000, Actual data_out from design = 00000000000000
# - At time=15 Computed From Testbench : data_in = 00000000, expected parity = 0000, expected overall_parity = 0, expected ecc_out = 00000000000000
#
# *Test Passed* : data_out generated from design matches data_out generated from tesbench!
# - At time=30 For Input data_in : 10011001, Expected data_out from testbench = 1001010001011, Actual data_out from design = 1001010001011
# - At time=30 Computed From Testbench : data_in = 10011001, expected parity = 0001, expected overall_parity = 1, expected ecc_out = 1001010001011
#
# *Test Passed* : data_out generated from design matches data_out generated from tesbench!
# - At time=45 For Input data_in : 00011101, Expected data_out from testbench = 0001111001111, Actual data_out from design = 0001111001111
# - At time=45 Computed From Testbench : data_in = 00011101, expected parity = 1011, expected overall_parity = 1, expected ecc_out = 0001111001111
#
# *Test Passed* : data_out generated from design matches data_out generated from tesbench!
# - At time=60 For Input data_in : 11010010, Expected data_out from testbench = 1101100100111, Actual data_out from design = 1101100100111
# - At time=60 Computed From Testbench : data_in = 11010010, expected parity = 1011, expected overall_parity = 1, expected ecc_out = 1101100100111
#
# *Test Passed* : data_out generated from design matches data_out generated from tesbench!
# - At time=75 For Input data_in : 01100111, Expected data_out from testbench = 0110001101010, Actual data_out from design = 0110001101010
# - At time=75 Computed From Testbench : data_in = 01100111, expected parity = 0001, expected overall_parity = 0, expected ecc_out = 0110001101010
#
# *Test Passed* : data_out generated from design matches data_out generated from tesbench!
# - At time=90 For Input data_in : 11001000, Expected data_out from testbench = 1100010000001, Actual data_out from design = 1100010000001
# - At time=90 Computed From Testbench : data_in = 11001000, expected parity = 0000, expected overall_parity = 1, expected ecc_out = 1100010000001
#
# *Test Passed* : data_out generated from design matches data_out generated from tesbench!
# - At time=105 For Input data_in : 01011010, Expected data_out from testbench = 0101010100000, Actual data_out from design = 0101010100000
# - At time=105 Computed From Testbench : data_in = 01011010, expected parity = 0000, expected overall_parity = 0, expected ecc_out = 0101010100000
#
a. #
```

3. Code snapshot (Student Added Part)

```
// Create binary value from 0 until N (i.e. binary_value[i] = N)
// This binary_value array is later required in the code to iterate to generate parity[0],
// parity[1],parity[2],parity[3]
for(int i=0; i<=N; i++) begin
    // Student to add code here to fill binary_value array each index with "i"
    for (int b = 1; b <= R; b++) begin
        binary_value[i][b] = (i >> (b-1)) & 1;
    end
end
```

a. end You, 4 seconds ago • Uncommitted changes

```
// Store input data in ecc code word
// Note : use for loop to insert din data bits in ecc_code
// Iterate using for loop ecc_code packed array and skip power of 2 positions
// in ecc_code packed array using if(e_idx != 2**$clog2(e_idx))
// Student to update and fill in the code below
for(int e_idx=1; e_idx <= N; e_idx++) begin // for loop line needs to be completed by student
    if ((e_idx & (e_idx - 1)) != 0) begin
        ecc_code[e_idx] = din[d_idx];
        d_idx++;
    end else begin
        ecc_code[e_idx] = 0;
    end
end
```

b. end

```
// Generate parity values
// Hint : Nested for loop required to iterate over binary_value table
// Remember : Below mentioned are hard coded formulas for each parity bit for M=8, N=4
// But instead of having these hard coded formula's use for loop to iterate over
// binary_value two dimensional array and generate parity value.
//parity[0] = din[0] ^ din[1] ^ din[3] ^ din[4] ^ din[6];
//parity[1] = din[0] ^ din[2] ^ din[3] ^ din[5] ^ din[6];
//parity[2] = din[1] ^ din[2] ^ din[3] ^ din[7];
//parity[3] = din[4] ^ din[5] ^ din[6] ^ din[7];
for(int i=1; i<=R; i++) begin
    parity[i-1] = 0;
    for(int j=1; j<=N; j++) begin
        if (binary_value[j][i] == 1) begin
            parity[i-1] ^= ecc_code[j];
        end
    end
end
```

c. end

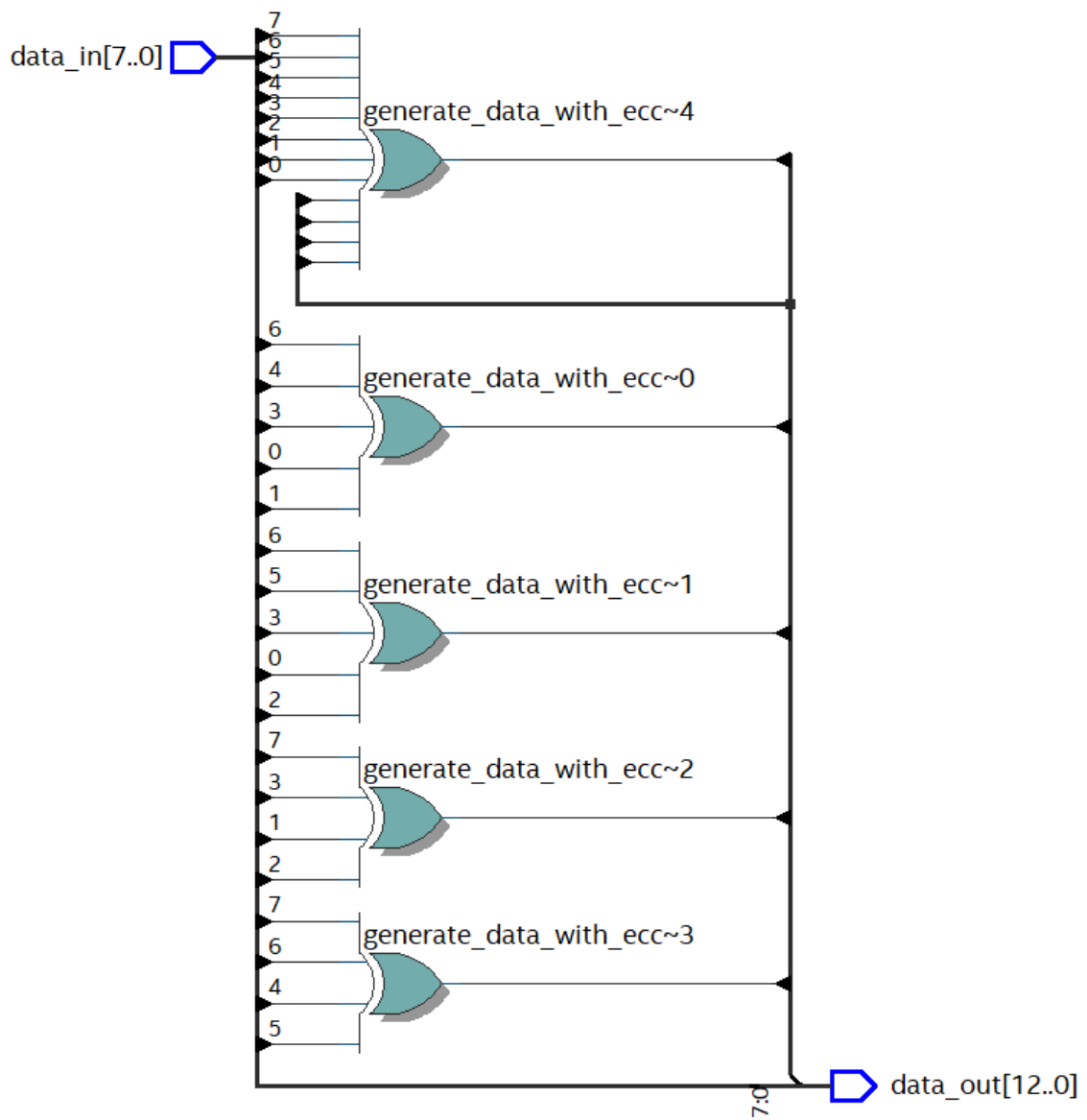
```
// Store parity in ecc code word
// Using for loop iterate ecc_code and store parity value generated
// from above mentioned for loop in power of 2 positions.
// For power of two indexing, 2**i can be used
for (int i=0; i<R; i++) begin // student to complete this line
    // Student to add code here
    ecc_code[1<<i] = parity[i];
end

// Compute overall parity of code word
// when computing overall parity use ecc_code variable which has
// original message bits along with computed parity bits
// Student to add code to compute overall parity
// <your code here using assign statement or any other approach>
// student to complete this code
overall_parity = 0;
for (int j=1; j<=N; j++) begin
    overall_parity ^= ecc_code[j];
end

// Generate data with ecc code and overall parity and return this value
// Overall parity is added in LSB of final data out
// Note : ecc_code should be upper bits and overall_parity should be LSB bit
generate_data_with_ecc = {ecc_code[N:1], overall_parity}; //student to add code here
```

d.

4. Post synthesis RTL netlist viewer snapshot



a.

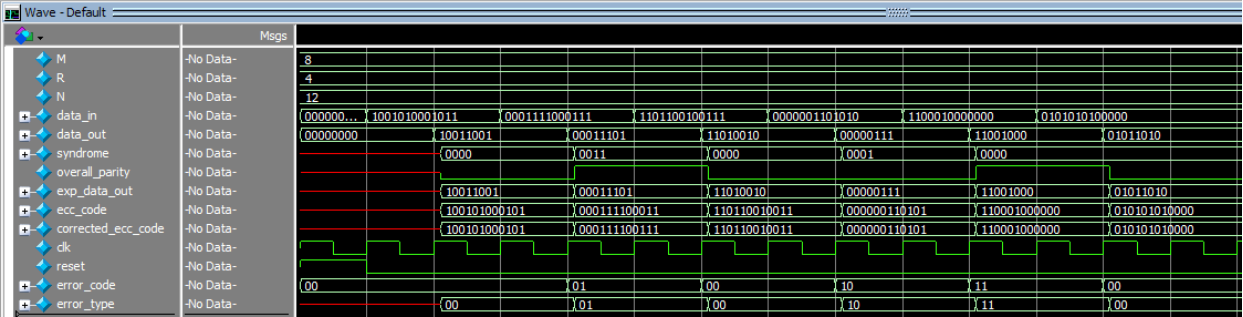
5. FPGA resource usage snapshot

Analysis & Synthesis Resource Usage Summary		
 <<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	3
2		
3	▼ Combinational ALUT usage for logic	5
1	-- 7 input functions	0
2	-- 6 input functions	1
3	-- 5 input functions	2
4	-- 4 input functions	2
5	-- <=3 input functions	0
4		
5	Dedicated logic registers	0
6		
7	I/O pins	21
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	data_in[2]~input
12	Maximum fan-out	4
13	Total fan-out	58
a. 14	Average fan-out	1.23

ECC Decoder

1. Simulation waveform

a.



2. Modelsim transcript snapshot

```
VSDM 16> run -all
# "Test Passed" : Both data_out and error_code generated from design matches data_out and error_code generated from tesbench!
# - At time=21 Computed From design for input data_in : 1001010001011, Expected data_out from testbench = 10011001, Actual data_out from design = 10011001
# - At time=21 Computed From design for input data_in : 1001010001011, Expected error_code from testbench = 00, Actual error_code from design = 00, syndrome = 0000, overall_parity = 0
#
# "Test Passed" : Both data_out and error_code generated from design matches data_out and error_code generated from tesbench!
# - At time=41 Computed From design for input data_in : 0001111000111, Expected data_out from testbench = 00011101, Actual data_out from design = 00011101
# - At time=41 Computed From design for input data_in : 0001111000111, Expected error_code from testbench = 01, Actual error_code from design = 01, syndrome = 0011, overall_parity = 1
#
# "Test Passed" : Both data_out and error_code generated from design matches data_out and error_code generated from tesbench!
# - At time=61 Computed From design for input data_in : 1101100100111, Expected data_out from testbench = 11010010, Actual data_out from design = 11010010
# - At time=61 Computed From design for input data_in : 1101100100111, Expected error_code from testbench = 00, Actual error_code from design = 00, syndrome = 0000, overall_parity = 0
#
# "Test Passed" : Both data_out and error_code generated from design matches data_out and error_code generated from tesbench!
# - At time=81 Computed From design for input data_in : 0000001101010, Expected data_out from testbench = 00000111, Actual data_out from design = 00000111
# - At time=81 Computed From design for input data_in : 0000001101010, Expected error_code from testbench = 10, Actual error_code from design = 10, syndrome = 0001, overall_parity = 0
#
# "Test Passed" : Both data_out and error_code generated from design matches data_out and error_code generated from tesbench!
# - At time=101 Computed From design for input data_in : 1100010000000, Expected data_out from testbench = 11001000, Actual data_out from design = 11001000
# - At time=101 Computed From design for input data_in : 1100010000000, Expected error_code from testbench = 11, Actual error_code from design = 11, syndrome = 0000, overall_parity = 1
#
# "Test Passed" : Both data_out and error_code generated from design matches data_out and error_code generated from tesbench!
# - At time=121 Computed From design for input data_in : 0101010100000, Expected data_out from testbench = 01011010, Actual data_out from design = 01011010
# - At time=121 Computed From design for input data_in : 0101010100000, Expected error_code from testbench = 00, Actual error_code from design = 00, syndrome = 0000, overall_parity = 0
#
```

a.

3. Code snapshot (Student Added Part)

```
always_comb begin
    // Re-compute overall parity of incoming data_in
    // Remember to discard LSB bit in incoming data_in[0] to re-compute overall parity
    // student to complete this code
    overall_parity = 0;
    for (int j=1; j<=N; j++) begin
        overall_parity ^= data_in[j];
    end

    // Remove LSB bit from incoming data to generate ecc code word
    // LSB bit in incoming data represents overall_parity sent by the ecc_encoder
    ecc_code = data_in[N:1]; // student to complete this code

    // Generate Syndrome
    syndrome = generate_syndrome(ecc_code);
```

a.

```
// Generate error type and final data out based on syndrome and overall parity check
// use if-else-if statements to generate error_type (i.e. error code)
// This error_type value indicates if its single bit error, double bit error, overall parity bit error
// See below definition of error_type (i.e. error_code) which needs to be implemented
// single bit error : if syndrome!=0 and overall_parity re-computed != incoming overall parity data_in[0] then error_type = 2'b01
// No error : if syndrome==0 and overall_parity re-computed == incoming overall parity data_in[0] then error_type = 2'b00
// overall parity bit error : if syndrome==0 and overall_parity re-computed != incoming overall parity data_in[0] then error_type = 2'b11
// double bit error : if syndrome!=0 and overall_parity re-computed == incoming overall parity data_in[0] then error_type = 2'b10
if((syndrome != 0) && (overall_parity != data_in[0])) begin
    //student to add code here // single bit error
    error_type = 2'b01;
end
else if((syndrome != 0) && (overall_parity == data_in[0])) begin
    //student to add code here // single bit error// double bit error
    error_type = 2'b10;
end
else if((syndrome == 0) && (overall_parity != data_in[0])) begin
    //student to add code here // single bit error // overall parity bit error
    error_type = 2'b11;
end
else begin
    error_type = 2'b00; // no error
end
```

b.

```
// Register message data and error type
// i.e. Added flipflop at the output ports data_out and error_code
always_ff@(posedge clk, posedge reset) begin
    // Student to add code here
    if (reset) begin
        data_out <= '0;
        error_code <= 2'b00;
    end else begin
        data_out <= message_data;
        error_code <= error_type;
    end
end
```

c.

```
// Function to create parity, ecc code, overall parity and final data out
function logic[R-1:0] generate_syndrome(logic[N:1] ecc_code_word);
begin
    // initialize return variable to 0
    generate_syndrome = 0;

    // Create binary value from 0 until N
    for(int i=0; i<=N; i++) begin
        // Student to add code here
        for (int b=1; b<=R; b++) begin
            binary_value[i][b] = (i >> (b-1)) & 1;
        end
    end

    // Generate syndrome values using ecc_code_word
    for(int i=1; i<=R; i++) begin
        for(int j=1; j<=N; j++) begin
            if((binary_value[j][i] == 1)) begin
                // Student to add code here
                generate_syndrome[i-1] ^= ecc_code_word[j];
            end
        end
    end
end
endfunction
```

d.


```

// Function to extract message data from ecc code
function logic[M-1:0] extract_message_data_from_ecc_code(logic[N:1] ecc_code_word);
begin
    // Declare and initialize variables
    integer d_idx;
    d_idx=0;

    // Extract message data from ecc code word and store it in extract_message_data_from_ecc_code[d_idx++]
    // Remember extract_message_data_from_ecc_code is the return variable which is the function name itself
    // data bit positions are non-power of two so ensure index of ecc_code_word is != 2**$clog2(e_idx)
    for (int e_idx=1; e_idx<=N; e_idx++) begin
        if (e_idx != (1 << $clog2(e_idx))) begin
            // Student to add code here
            extract_message_data_from_ecc_code[d_idx] = ecc_code_word[e_idx];
            d_idx++;
        end
    end
end
endfunction

```

e.

```

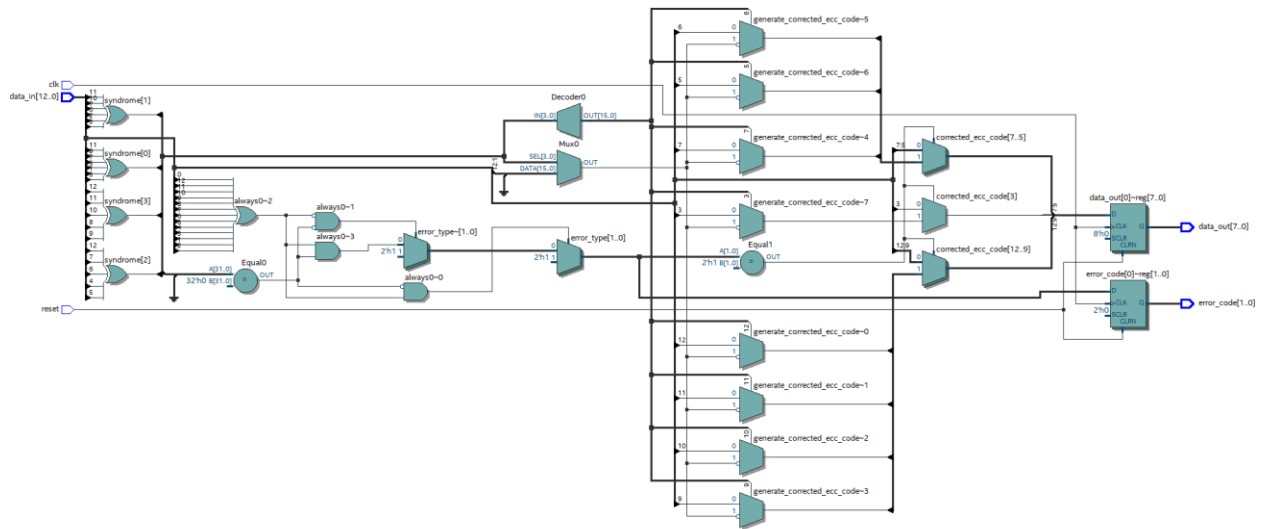
// Function to generate corrected ecc code word
function logic[N:1] generate_corrected_ecc_code(logic[N:1] ecc_code_word, logic[M-1:0] syn, logic[1:0] err_type);
begin
    // Declare and initialize variables
    generate_corrected_ecc_code = ecc_code_word;

    // Generate corrected ecc code word in case of single bit error (i.e. for error code 2'b01)
    // In case of no error, double bit error, overall parity bit error do not correct the data as data does not need correction
    if(err_type == 2'b01) begin // check here value of "err_type" coming into this function if its value is 2'b01 which is single bit error
        // Flip bit in ecc code based on syndrome value
        // Student to add code here
        generate_corrected_ecc_code[syn] = ~ecc_code_word[syn];
    end
    else begin
        // Do not flip bit
        // Student to add code here
    end
end
endfunction

```

f.

4. Post synthesis RTL netlist viewer snapshot



a.

5. FPGA resource usage snapshot

Analysis & Synthesis Resource Usage Summary		
<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	22
2		
3	▼ Combinational ALUT usage for logic	30
1	-- 7 input functions	0
2	-- 6 input functions	14
3	-- 5 input functions	3
4	-- 4 input functions	1
5	-- ≤3 input functions	12
4		
5	Dedicated logic registers	10
6		
7	I/O pins	25
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	syndrome[0]~1
12	Maximum fan-out	14
13	Total fan-out	199
14	Average fan-out	2.21

a.